

COURSE CODE/TITLE: 010-CPE21S1 - Data Structures and Algorithms

SECTION: CPE21S1

DATE PERFORMED: July 28 2026

DATE SUBMITTED: July 30 2026

NAME: Mendoza, Roy Gabriel I.

INSTRUCTOR: Engr. Jimlord Quejado

ACTIVITY NO. 4.1 Stacks

1. PROCEDURE OUTPUT

What procedures did you perform?

Table 4-1. Output of ILO A

```
1 //Tests the push, empty, size, pop, and top methods of the stack library.
2 #include <iostream>
3 #include <stack> // Calling Stack from the STL
4 using namespace std;
5 int main() {
6
7     stack<int> newStack;
8     newStack.push(3); //Adds 3 to the stack
9     newStack.push(8);
10    newStack.push(15);
11
12    // returns a boolean response depending on if the stack is empty or not
13    cout << "Stack Empty? " << newStack.empty() << endl;
14    // returns the size of the stack itself
15    cout << "Stack Size: " << newStack.size() << endl;
16    // returns the topmost element of the stack
17    cout << "Top Element of the Stack: " << newStack.top() << endl;
18    // removes the topmost element of the stack
19    newStack.pop();
20    cout << "Top Element of the Stack: " << newStack.top() << endl;
21    cout << "Stack Size: " << newStack.size() << endl;
22    return 0;
23 }
```

```
C:\Users\royga\CLionProjects\untitled1\keyufg.exe
is stack empty? 1
The top of the element is 15
is stack empty? 0
The size of the stack is 3
Removing elements from the stack
The top of the element is 8
The top of the element is 3
```

Code Analysis:

On this part of the activity, we have been able to demonstrate the use of the stack and its functions such as the push, pop, top, size, isEmpty, isFull, and all of which are important part of stack

Table 4-2. Output of ILO B.1

Code:

```

1  ● #include <iostream>
2
3  const size_t maxCap = 100;
4  int stack[maxCap];
5  int top = -1;
6  int newData, choice, i;
7
8  ↵ void push();
9  ↵ void pop();
10 ↵ void Top();
11 ↵ bool isEmpty();
12 bool isFull();
13 ↵ void prinAll();
14
15 ▢ ∨ int main() {
16     //=====
17     std::cout << "Enter the maximum number of elements (1-100): ";
18     std::cin >> i;
19     //=====
20     // Validate the user input
21
22     if (i <= 0 || i > maxCap) {
23         std::cout << "Invalid! Maximum allowed is " << maxCap << ".\n";
24         return 0;
25     }
26     //=====
27     while (true) {
28         std::cout << "\n==== STACK OPERATIONS =====\n";
29         std::cout << "1. PUSH\n" << "2. POP\n" << "3. TOP\n" << "4. isEmpty\n" << "5. PRINT CONTENTS\n" << "6. EXIT\n" << "Choose your choice: ";
30         //=====
31         std::cin >> choice;
32         switch (choice) {
33             case 1:
34                 push();
35                 break;
36             case 2:
37                 pop();
38                 break;
39             //=====
40             case 3:

```

```

41         Top();
42         break;
43         //=====
44     case 4:
45         if (isEmpty())
46             std::cout << "Stack is Empty.\n";
47         else
48             std::cout << "Stack is NOT Empty.\n";
49         break;
50         //=====
51     case 5:
52         std::cout<<"Print out all the contents of the stack.\n";
53         prinAll();
54         break;
55         //=====
56     case 6:
57         std::cout << "Exiting program.\n";
58         return 0;
59         break;
60         //

```

```

61         return 0;
62         break;
63         //=====
64     default:
65         std::cout << "Invalid choice!\n";
66         break;
67     }
68     return 0;
69 }
70 //=====
71 //=====
72 //checks if the stack is empty
73 → bool isEmpty() {
74     if (top== -1) return true;
75     return false;
76 }
77 //=====
78 //adds new content to the very top

```

```

78 → void push() {
79     if (top==i-1) {
80         std::cout << "Stack Overflow! Cannot push more elements.\n";
81         return;
82     }
83     std::cout << "Enter value to push: ";
84     std::cin >> newData;
85     stack[++top] = newData;
86     std::cout << newData << " pushed into the stack.\n";
87 }
88 //=====
89 //removes the topmost content
90 → void pop() {
91     if (isEmpty()) {
92         std::cout << "Stack Underflow! Stack is empty.\n";
93         return;
94     }
95     std::cout << "Popping: " << stack[top] << std::endl;
96     top--;
97 }
98 //=====

```

```

    //shows the latest and last in
100 → void Top() {
101     if (isEmpty()) {
102         std::cout << "Stack Underflow! Stack is empty.\n";
103         return;
104     }
105     std::cout << "Top element: " << stack[top] << std::endl;
106 }
107 //=====
108 // to print all the contents of the stack
109 //checking if the stack is empty
110 → void prinAll() {
111     if (isEmpty()) {
112         std::cout << "Stack is empty.\n";
113         return;
114     }
115     //literal printing of the stack
116     std::cout << "Stack elements (top to bottom):\n";
117     for (int j = top; j >= 0; --j) {
118         std::cout << "|" << stack[j] << "|" << std::endl;
119     }

```

Output:

```
===== STACK OPERATIONS =====
```

```
1. PUSH
```

```
2. POP
```

```
3. TOP
```

```
4. isEmpty
```

```
5. PRINT CONTENTS
```

```
6. EXIT
```

```
Choose your choice:1
```

```
Enter value to push:5
```

```
5 pushed into the stack.
```

```
===== STACK OPERATIONS =====
```

1. PUSH
2. POP
3. TOP
4. isEmpty
5. PRINT CONTENTS
6. EXIT

Choose your choice:1

Enter value to push:10

10 pushed into the stack.

```
===== STACK OPERATIONS =====
```

1. PUSH
2. POP
3. TOP
4. isEmpty
5. PRINT CONTENTS
6. EXIT

Choose your choice:1

Enter value to push:15

15 pushed into the stack.

```
===== STACK OPERATIONS =====
```

1. PUSH
2. POP
3. TOP
4. isEmpty
5. PRINT CONTENTS
6. EXIT

Choose your choice:1

Enter value to push:20

20 pushed into the stack.

```
===== STACK OPERATIONS =====
```

1. PUSH
2. POP
3. TOP
4. isEmpty
5. PRINT CONTENTS
6. EXIT

Choose your choice:3

Top element: 20

```
===== STACK OPERATIONS =====
```

```
1. PUSH
```

```
2. POP
```

```
3. TOP
```

```
4. isEmpty
```

```
5. PRINT CONTENTS
```

```
6. EXIT
```

```
Choose your choice:4
```

```
Stack is NOT Empty.
```

```
===== STACK OPERATIONS =====
```

```
1. PUSH
```

```
2. POP
```

```
3. TOP
```

```
4. isEmpty
```

```
5. PRINT CONTENTS
```

```
6. EXIT
```

```
Choose your choice:5
```

```
Print out all the contents of the stack.
```

```
Stack elements (top to bottom):
```

```
|20|
```

```
|15|
```

```
|10|
```

```
|5|
```

```
===== STACK OPERATIONS =====
```

1. PUSH
2. POP
3. TOP
4. isEmpty
5. PRINT CONTENTS
6. EXIT

Choose your choice:2

Popping: 20

```
===== STACK OPERATIONS =====
```

1. PUSH
2. POP
3. TOP
4. isEmpty
5. PRINT CONTENTS
6. EXIT

Choose your choice:5

Print out all the contents of the stack
Stack elements (top to bottom):

|15|
|10|
|5|

Code Analysis:

In this part of the activity, we are tasked to use an array for a stack, where we also apply what we did on the first task, but here we must be able to remove and add and show the final content of the stack if it is not empty. The use of **Push()**, the function that adds a new content to the very top of the array, was helpful in the creation or one of the most important functions to have the contents of the stack. Both the **isEmpty**, and **isFull**, is important at a certain scenario to know if the array stack is already full or empty. The **Top()** is basically outputting or showing the most latest or the content that is inputted the last. Then we have the **Pop()** one of the important functions for it removes the content or removes an element of the array stack from the stack itself. Lastly, the student made a function of **prinAll()**, which just basically shows the contents of the entire stack without disturbing the distribution of the contents.

OUTPUT OF ILO B.2

Runner CODE:

```
1  #include <iostream>
2  #include "STACKS.h"
3
4  int main() {
5      std::cout << "Pushing 'c', 'p', 'e' ONTO THE STACK\n";
6
7      // pushing characters
8      push<char>( newData: 'c');
9      push<char>( newData: 'p');
10     push<char>( newData: 'e');
11
12     // display all the contents
13     displayAll<char>();
14
15     // display the current top
16     top<char>();
17
18     // popping
19     std::cout << "Popping FROM THE STACK\n";
20
21     while (!isEMPT<char>()) {
22         std::cout << "Popped: " << pop<char>() << std::endl;
23     }
24
25     std::cout << "Testing the Underflow \n";
26     pop<char>();
27
28     return 0;
29 }
```

Header:	EXPLANATIONS
<pre> 1 #ifndef UNTITLED1_STACKS_H 2 #define UNTITLED1_STACKS_H 3 #include <iostream> 4 #include <ostream> 5 6 template <typename T> 7 class Node { 8 public: 9 T data; // value we need to store 10 Node* next = nullptr; // pointer to the next node, defaults to null 11 }; 12 13 //create a global pointer for the top of the stack 14 template <typename T> 15 Node<T> *head = nullptr; 16 17 //===== 18 //operations for the stack 19 //===== 20 //push 21 template <typename T> 22 void push(T newData) { 23 //create a new node and assign the newData 24 Node<T>* newNode = new Node<T>; 25 newNode->data = newData; 26 27 //new node points to current top, then becomes the new top 28 newNode->next = head<T>; 29 head<T> = newNode; 30 } 31 </pre>	>> Creating a Public class for storing and pointing it to null. >>PUSH(): this function creates a new node to store the data and is assigned as the new, which then replaces the old node and pushes it back, meaning that the first node is now the second, and the new is the new Node.

```

32 //pop
33 template <typename T>
34 T pop() {
35     //stopping if the stack is empty
36     if (head<T> == nullptr) {
37         std::cout << "Stack underflow!" << std::endl;
38         return T();
39     }
40
41     //save the current top node
42     Node<T>* temp = head<T>;
43
44     //store its data
45     T tempVal = temp->data;
46
47     //move the head to the next node
48     head<T> = head<T>->next;
49
50     //free the removed node
51     delete temp;
52
53     //return the removed value
54     return tempVal;
55 }
56

```

```

57 //isEmpty
58 template <typename T>
59 bool isEmpty() {
60     return head<T> == nullptr;
61 }

```

>>POP(): the first part of this function is to check if there is content in the node, or that there is something to be printed out.

The next part is the new top node is put in temp to be deleted later. But the head node is moved to the next one to keep the 2nd node as the new head node.

>>isEmpty(): this function basically just check if the head node is null or in basic terms, there are no content in the list.

```

63 //top
64 template <typename T>
65 void top() {
66     //create error catching:
67     if (isEmpty<T>()){
68         std::cout << "Stack underflow" << std::endl;
69         return;
70     }
71     //display the top node
72     std::cout << "Top of stack is: "<< "|" << head<T>->data << "|" << std::endl;
73 }
74

```

>> **TOP():** this function uses the work of the **isEmpty()** to check if there are contents to the nodes. If there are, It then displays the head node which is the most top content.

```

75 //displayAll
76 template <typename T>
77 void displayAll() {
78     if (isEmpty<T>()) {
79         std::cout << "Stack is empty" << std::endl;
80         return;
81     }
82     std::cout << "Stack contents (top to bottom): ";
83     Node<T>* temp= head<T>;
84     while (temp !=nullptr) {
85         std::cout << "\n|" << temp->data << "|" ";
86         temp = temp->next;
87     }
88     std::cout << std::endl;
89 }
90

```

>> **displayALL():** this function also uses the **isEmpty()** to check if there are contents on the list, if there are, it then use the while loop to sequentially print out the contents of the head, and move the head pointer to the next node, until it meets the nullptr.

OUTPUT:

```
"C:\Users\royga\CLionProjects\untitled1\Stack Using Linked Lists.exe"  
Pushing 'c', 'p', 'e' ONTO THE STACK  
Stack contents (top to bottom):  
|e|  
|p|  
|c|  
Top of stack is: |e|  
Popping FROM THE STACK  
Popped: e  
Popped: p  
Popped: c  
Testing the Underflow  
Stack underflow!  
  
Process finished with exit code 0
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Supplementary INSTRUCTIONS:

ILO C: Solve problems using an implementation of stack:

The following problem definition and algorithm is provided for checking the balancing of symbols. Create an implementation using stacks. Your output must include the following:

- Stack using Arrays
- Stack using Linked Lists
- (Optional) Stack using C++ STL

Problem Definition:

Stacks can be used to check whether the given expression has balanced symbols. This algorithm is very useful in compilers. Each time the parser reads one character at a time. If the character is an opening delimiter such as (, {, or [- then it is written to the stack. When a closing delimiter is encountered like), }, or]-the stack is popped. The opening and closing delimiters are then compared. If they match, the parsing of the string continues. If they do not match, the parser indicates that there is an error on the line.

Steps:

- Create a Stack.
- While(end of input is not reached) {
 - If the character read is not a symbol to be balanced, ignore it.
 - If the character is an opening symbol, push it onto the stack.
 - If it is a closing symbol:
 - Report an error if the stack is empty.
 - Otherwise, pop the stack.
 - If the symbol popped is not the corresponding opening symbol, report an error.
- At the end of input, if the stack is not empty: report an error.

Self-Checking:

For the following cases, complete the table using the code you created.

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)			
((A+B)+(C-D)			
((A+B)+[C-D])			
((A+B)+[C-D])			

CODE:

OUTPUT:

Progress report:

>> July 30 2026: on this day, I was able to finish the the activity ILO B, and have already started with the Supplementary, although I was not able to finish the Supplementary activity since the lack of time and the not being able to fully understand the true goal of the activity unlike the previous parts.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.



I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.